

State Institute of Technology (SIT)

CS101: Midterm Exam - Data Structures & Algorithms

Student Name: Siddharth Sen

Roll Number: CS2026-092

Session: Fall 2026 Semester

Date: August 3, 2026

Question 1 (10 Marks): Define a Binary Search Tree (BST) and write its search complexity. Explain why search complexity can degenerate to $O(N)$ and how height-balanced trees (AVL/Red-Black) solve this.

Answer 1:

A Binary Search Tree (BST) is a node-based binary tree data structure with the following recursive properties:

1. The left subtree of a node contains only nodes with keys less than the node's key.
2. The right subtree of a node contains only nodes with keys greater than the node's key.
3. Both the left and right subtrees must also be binary search trees.

Search Complexity:

- Best and Average Case: $O(\log N)$, where N is the number of nodes. In a balanced BST, the height of the tree is bounded by $\log N$, meaning search path halves at each level.
- Worst Case: $O(N)$. This occurs when the tree becomes unbalanced. For example, if elements are inserted in a pre-sorted order (e.g., 1, 2, 3, 4, 5), the tree degenerates into a linear chain (known as a skewed tree), resembling a linked list. In this state, searches require linear scanning.

Height-Balanced Solution (AVL):

AVL trees solve this degeneration problem by enforcing structural balance constraints. Specifically, they maintain a balance factor $BF = |\text{height}(\text{Left Subtree}) - \text{height}(\text{Right Subtree})| \leq 1$ for every node. If an insertion or deletion violates this property ($BF > 1$), AVL trees trigger rebalancing rotations (Single LL/RR, or Double LR/RL rotations) to reduce height. This guarantees that the height h remains strictly bounded by $O(\log N)$, keeping lookup complexity consistently at $O(\log N)$.

Question 2 (10 Marks): Write the pseudo-code for inserting a node in a BST. Explain the algorithm's time and space complexity.

Answer 2:

Here is the recursive logic for inserting a key value in a Binary Search Tree:

```
Node insert(Node root, int val) {
    // Base Case: return a new node if tree is empty
    if (root == null) {
        return new Node(val);
    }

    // Recursive traversal directions
    if (val < root.key) {
        root.left = insert(root.left, val);
    } else if (val > root.key) {
        root.right = insert(root.right, val);
    }

    // Duplicate values are ignored in this implementation
    return root;
}
```

Complexity Explanations:

- *Time Complexity: $O(h)$, where h is the tree height. If the tree is height-balanced, h is $O(\log N)$. If the tree degenerates into a skewed structure, h is $O(N)$ as it traverses all nodes.*
- *Space Complexity: $O(h)$ auxiliary stack space. Because the code is recursive, each call places a stack frame in memory. In worst-case skewed trees, this requires $O(N)$ stack frames. (An iterative implementation would require $O(1)$ space).*